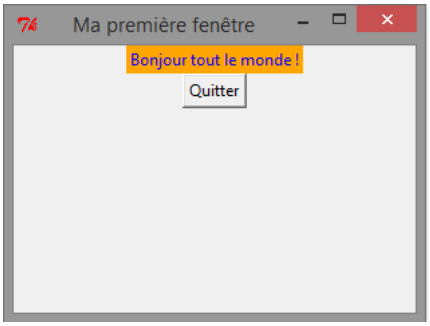
TKINTER

1 Premiers pas

Création d'une fenêtre avec deux widgets (contraction de window gadget) : un bout de texte (label) et un bouton (button).

```
from tkinter import*
fen1=Tk()
fen1.title('Ma première fenêtre')
fen1.geometry('300x200')
tex1=Label(fen1,text='Bonjour tout le monde !', fg='blue',  bg='orange')
bou1=Button(fen1,text='Quitter',command=fen1.destroy)
tex1.pack()
bou1.pack()
fen1.mainloop()
```



fen1=Tk() crée la fenêtre qui s'appellera fen1.

tex1=Label(fen1, text='Bonjour tout le monde !', fg='blue', bg='orange') : le premier argument est le nom de la fenêtre dans lequel sera le bouton (fen1 est le widget maître de l'objet tex1, ou l'objet tex1 est un widget esclave de l'objet fen1). Le deuxième est facile à comprendre, le troisième est la couleur d'avant plan (foreground, en anglais) et le quatrième est la couleur de fond (background).

tex1.pack() : nous activons ici la méthode pack() à l'objet tex1. Cette méthode agit sur la disposition géométrique, la fenêtre maître est réduite automatiquement pour qu'elle soit juste assez grande pour contenir les widgets esclaves.
Un objet pour être visible doit être mis en place c'est ce que fait la méthode pack(), il y a aussi la méthode grid() que l'on verra plus loin

fen1.mainloop() : c'est cette ligne qui provoque le démarrage du réceptionnaire d'événements associé à la fenêtre. Cette instruction est nécessaire pour que l'application soit ń à l'affût ż des clics de souris, des pressions exercées sur les touches du clavier, etc. C'est donc cette instruction qui la met en marche.

Contrôle de la fenêtre
fen1.title('nom') : définir le nom de la fenêtre
fen1.maxsize(width=,height=) : taille maximale autorisée
fen1.minsize(width=,height=) : taille minimale autorisée
fen1.resizable(width=YES/NO,height=YES/NO) : autorisation de redimensionnement

La structure de base d'un programme simple utilisant Tkinter est globalement (et dans cet ordre) :

- importation du module tkinter
- définition des fonctions qui sont les commandes
- définition de la fenêtre principale
- définition des widgets
- positionnement des widgets
- lancement de l'application avec mainloop.

Quelques widgets

- Button : un bouton à cliquer, provoquer l'exécution d'une commande
- Canvas : un espace pour disposer divers éléments graphiques
- Entry : un champ de saisie (saisie au clavier)
- Label : permet d'afficher un texte quelconque ou une image.
- Text : zone d'affichage de texte(2D)
- Frame : cadre pour positionner d'autres widgets

Dans la définition de ces widgets, figure obligatoirement comme premier élément, le nom d'un widget les contenant : par exemple la fenêtre définie à l'appel de Tk() ou un autre widget comme Frame.

Tous les autres paramètres sont optionnels.....
On peut avoir : une couleur d'écriture, une couleur de fond, une taille, une commande associée .....

Toutes ces options ont une valeur par défaut (l'écriture par exemple est noire, la taille s'adapte au contenu...)

Pour connaitre les méthodes et options d'un widget par exemple Button, il suffit de taper dans la console : print dir(Button())

Quelques options communes aux widgets

- width : valeur positive indiquant la largeur du widget en unité de caractère (donc en pixel pour certains et en caractères dans la police donnée par l'option font pour d'autres) .
- height : valeur positive indiquant la hauteur du widget .Doit valoir au moins 1. (pas de height pour Entry qui a forcément une hauteur d'une ligne). Pour une zone de texte c'est en lignes d'écriture (donc ne pas mettre trop grand), pour un canvas c'est en pixels)
- background (bg) : couleur de l'arrière plan du widget
- foreground (fg) : couleur du premier plan du widget c'est à dire de l'écriture (sauf Canvas)

2 Agencement

Un widget doit être positionné pour apparaître. Il ne suffit pas de le définir....

- Chaque widget doit être positionné pour apparaître (mais on peut les faire apparaître dans une commande au cours d'un programme)
- 2 méthodes principales : grid et pack
  Attention : il ne faut pas mélanger plusieurs méthodes d'agencement dans le même programme (sinon les widgets n'apparaissent pas)

2.1 grid

Ce gestionnaire positionne les widgets selon une grille 2D .
On indique dans quelle ligne (row) et dans quelle colonne (column) de ce tableau on souhaite placer les widgets.
Ce n'est pas absolu : on peut numéroter les lignes et les colonnes comme on veut, en partant de zéro, ou de un, ou encore d'un nombre quelconque : Tkinter ignorera les lignes et colonnes vides. Tkinter détermine automatiquement le nombre de lignes et de colonnes nécessaires.
Les objets sont positionnés dans l'ordre des lignes et des colonnes mais pas de façon absolue (il n'y a pas de case 1,2) mais quelque chose situé en colonne a est plus à gauche que quelque chose situé en colonne b si a<b (de même pour les lignes).
On peut étirer une case sur plusieurs lignes ou colonnes avec rowspan= nombre de lignes et columnspan=nombre de colonnes

2.2 pack

Ce gestionnaire de géométrie permet de positionner les widgets dans le widget parent selon 4 zones : LEFT, RIGHT, TOP et BOTTOM en utilisant side=BOTTOM par exemple comme argument de pack.

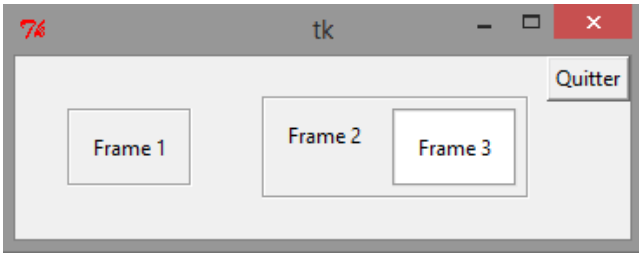
Les méthodes sont :

- pack(option=valeur,...)
- pack_forget() : depack un widget (le fait disparaitre)
- On a l'option principale : side qui spécifie le côté contre lequel le widget doit être 'paqué'.
  Pour paquer les widgets verticalement, utiliser TOP (valeur par défaut).
  Pour paquer les widgets horizontalement, utiliser LEFT. On peut aussi utiliser BOTTOM et RIGHT

2.3 Frame

Ce widget sert le plus souvent de 'conteneur' ou simplement comme délimiteur de position pour d'autres widgets.
Il peut posséder une bordure ou un fond coloré (possède par ailleurs les mêmes options que d'autres widgets déjà vus)
On va positionner un cadre (très souvent transparent), puis positionner d'autres widgets à l'intérieur. Les positions des widgets internes seront calculées dans ce cadre (et non pas par rapport à la fenêtre). Ceci permet notamment de regrouper des widgets.....

```
from tkinter import*
Fp=Tk()
f1= Frame(Fp, borderwidth=2, relief=GROOVE)
f1.pack(side=LEFT, padx=30, pady=30)
f2=Frame(Fp,borderwidth=2, relief=GROOVE)
f2.pack(side=LEFT, padx=10, pady=10)
f3 = Frame(f2, bg='white', borderwidth=2, relief=GROOVE)
f3.pack(side=RIGHT, padx=5, pady=5)
Label(f1, text='Frame 1').pack(padx=10, pady=10)
Label(f2, text='Frame 2').pack(padx=10, pady=10)
Label(f3, text='Frame 3',bg='white').pack(padx=10, pady=10)
Button(Fp,text='Quitter',command=Fp.destroy).pack()
Fp.mainloop()
```



3 Les boutons

Les boutons permettent de proposer une action à l'utilisateur.

Option	Type	Fonction
text	chaine	Teste affiché dans le libellé, sur une ou plusieurs lignes
image	image	image affichée dans le widget Button. Si cette option est précisée, elle prend le dessus sur l'option text
command	constante	méthode ou fonction appelée quand le bouton est appuyé pour appeler une fonction f qui est par exemple définie pour deux variables x et y il faut écrire : **command=lambda :** $f(2,3)$ si on veut l'appeler pour les valeurs $x=2$ et $y=3$
justify	constante	justification du texte : LEFT,RIGHT ou CENTER
anchor	constante	situation du texte ou image dans le libellé N,NE, E,SE,S,SW,W,NW ou CENTER (par défaut)
wraplength	écartement	détermine quand le texte du libellé doit être coupé. Cette option est donnée en unités d'écran. Par défaut, affichage sur une seule ligne
textvariable	variable	associe à une variable Tkinter (généralement StringVar) au libellé. Si la variable est modifiée, le texte du libellé est mis à jour
underline	entier	précise quel caractère est souligné dans le texte. -1 (par défaut) signifie qu'aucun n'est souligné.
padx,pady	distance	marges horizontale (padx) et verticale (pady) entre le texte ou l'image et le cadre du bouton
state	constante	état du bouton : NORMAL (par défaut), ACTIVE ou DISABLED

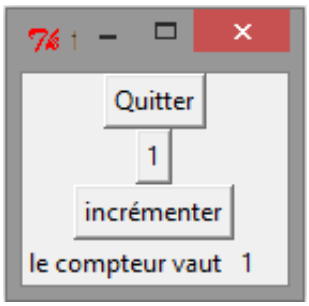
Ces options peuvent être choisies lors de la définition du bouton comme : bou1= Button(fen1, text='OUI') ou configurées au moment où on le souhaite à l'aide d'instructions : bou1.configure (fg=blue, text='NON')

Pour command=lambda : affiche(photo1) permet d'appeler une fonction qui a plusieurs variables Le text contenu dans un bouton peut être récupéré dans une variable spécifique de type StringVar().
StringVar() est une classe qui permet de définir des variables Tkinter, en fait des objets, qui se comportent comme des variables pour les widgets Tkinter. Pour accéder à sa valeur depuis Python, il faut les méthodes spécifiques de cette classe d'objets, set() pour lui assigner une valeur et get() pour la récupérer.

```
from tkinter import*
Fp=Tk()
compteur=StringVar()
compteur.set(0)

def undeplus() :
    global compteur
    compteur.set(int(compteur.get())+1)

bout1=Button(Fp,text='Quitter',command=Fp.destroy)
bout1.pack( )
bout2=Button(Fp,textvariable=compteur)
bout2.pack()
bout3=Button(Fp,text='incrémenter',command=undeplus )
bout3.pack()
lab1=Label(Fp,text='le compteur vaut ')
lab1.pack(side='left')
lab2=Label(Fp,textvariable=compteur)
lab2.pack(side='left')
Fp.mainloop()
```

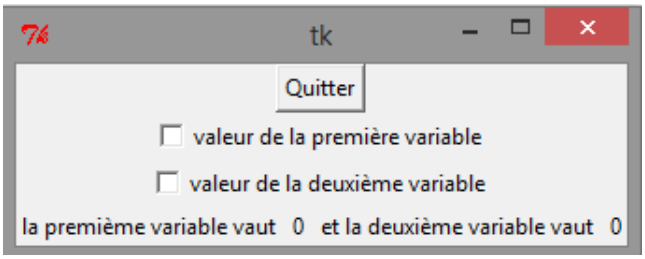


3.1 les cases à cocher

Dans l'exemple ci-dessous, les objets val1 et val 2 contiennent l'équivalent d'une variable de type chaine.
Les Checkbutton proposent à l'utilisateur de cocher une option

```
from tkinter import*
Fp=Tk()
val1=StringVar()
val2=StringVar()
val1.set(0)
val2.set(0)
```

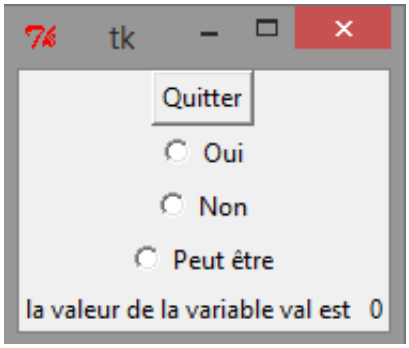
```
bout1=Button(Fp,text='Quitter',command=Fp.destroy)
bout1.pack(side='top')
chk1=Checkbutton(Fp,text='valeur de la première variable',variable=val1)
chk1.pack()
chk2=Checkbutton(Fp,text='valeur de la deuxième variable',variable=val2)
chk2.pack()
lab1=Label(Fp,text="la première variable vaut ")
lab1.pack(side='left')
lab12=Label(Fp,textvariable=val1)
lab12.pack(side='left')
lab2=Label(Fp,text=" et la deuxième variable vaut ")
lab2.pack(side='left')
lab22=Label(Fp,textvariable=val2)
lab22.pack(side='left')
Fp.mainloop()
```



3.2 les boutons radio

Radiobutton sont des cases à cocher dans un groupe et dans ce groupe seul un élément peut être sélectionné.

```
from tkinter import*
Fp=Tk()
val= StringVar()
val.set(0)
bout=Button(Fp,text='Quitter',command=Fp.destroy)
bout.pack(side='top')
bout1 = Radiobutton(Fp, text='Oui', variable=val, val=1)
bout2 = Radiobutton(Fp, text='Non', variable=val, val=2)
bout3 = Radiobutton(Fp, text='Peut être', variable=val,
    val=3)
bout1.pack()
bout2.pack()
bout3.pack()
lab1=Label(Fp,text='la valeur de la variable val est ')
lab1.pack(side='left')
lab2=Label(Fp,textvariable=val)
lab2.pack(side='left')
Fp.mainloop()
```



3.3 message d'alerte

Il faut le module messagebox

```
from tkinter import*
from tkinter.messagebox import *
def callback() :
        if askyesno('Titre 1', 'Etes-vous sûr de vouloir faire ça?') :
                showwarning('Titre 2', 'Tant pis...')
        else :
                showinfo('Titre 3', 'Vous avez peur!')
                showerror("Titre 4", "Aha")
```

```
Fp=Tk()
Button(Fp,text='Quitter',command=Fp.destroy).pack()
Button(Fp,text='Action', command=callback).pack()
Fp.mainloop()
```

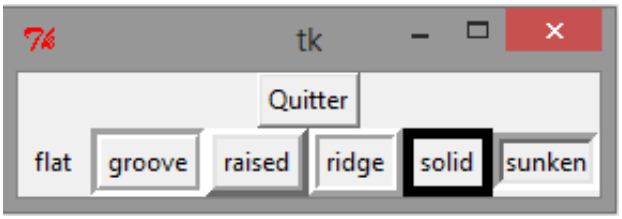
Les alertes possibles sont :
showinfo()
showwarning()
showerror()
askquestion()
askokcancel()
askyesno()
askretrycancel()

4 Relief et cadre

L'option "relief" détermine le type de bordure qui va encadrer le widget.
Les choix sont : flat, groove, raised, ridge, solid et sunken.

L'option "borderwidth" indique la largeur de la bordure en question.

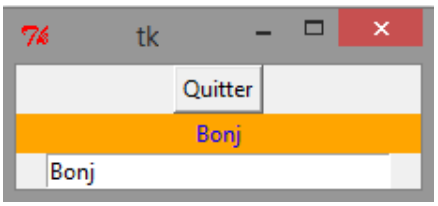
```
 from tkinter import*
fen1=Tk()
Button(fen1,text='Quitter',command=fen1.destroy).pack()
Button(fen1,text='flat',relief='flat',borderwidth=5).pack(side='left')
Button(fen1,text='groove',relief='groove',borderwidth=5).pack(side='left')
Button(fen1,text='raised',relief='raised',borderwidth=5).pack(side='left')
Button(fen1,text='ridge',relief='ridge',borderwidth=5).pack(side='left')
Button(fen1,text='solid',relief='solid',borderwidth=5).pack(side='left')
Label(fen1,text='sunken',relief='sunken',borderwidth=5).pack(side='left')
fen1.mainloop()
```



5 La zone de saisie : Entry

Ce widget permet la saisie d'un texte par l'utilisateur.
Dans l'exemple ci-dessous : le texte saisie est en même temps transcrit dans le label.

```
from tkinter import*
fen1=Tk()
Button(fen1,text='Quitter',command=fen1.destroy).pack()
saisie=StringVar()
Label(fen1,textvariable=saisie,width=30,fg='blue',bg='orange').pack()
Entry(textvariable=saisie,width=30).pack()
fen1.mainloop()
```



Il est possible de masquer le texte qui est saisi au moyen de la commande 'show' : show='x'
Ce widget permet d'afficher, mais surtout de saisir une ligne de texte qu'on pourra récupérer facilement ; ce widget possède une seule ligne.
On a les méthodes :
⋆ insert (index,text) : insère du texte au niveau de l'index spécifié. On utilisera insert(i,text) pour insérer du texte à l'index i et insert(END,text) pour ajouter en fin de ce qui est déjà écrit
⋆ delete (index), delete (from,to) : supprime le caractère situé à la position de l'index indiquée ou à l'intérieur d'un domaine donné.

⋆ On utilisera delete(0,END) pour effacer l'intégralité du texte du widget.
⋆ get ( ) : récupère le contenu du champ de saisie.
⋆ set(' ') : assigne une valeur

6 Le widget Text

Il nous servira essentiellement à afficher du texte

exemple : montexto=Text(lieu, width=400,height=3)
⋆ insert (index,text) : insère du texte à l'endroit donné (index) attention l'index correspond à ligne ET colonne donc du style 0.0
⋆ delete(index), delete(start,stop) : supprime le caractère de position "index" ou l'ensemble du texte situé entre start et stop. On peut par exemple effacer tout le contenu de Text en utilisant : montexto.delete('0.0',END)
⋆get(index), get(start,stop) : retourne le caractère de position "index" ou l'ensemble du texte situé entre start et stop

7 Les labels

Les labels sont des espaces prévus pour écrire du texte qui ne varie pas beaucoup

exemple : lab=Label(lieu, text='bonjour',fg ='red')

Si on veut modifier le texte écrit on peut utiliser : configure
lab.configure(text='au revoir')

lab.configure(text=' ') : permettra d'effacer ce qui a été écrit

Avec lab.configure() on peut aussi modifier n'importe quel autre paramètre du label.

Option du Label

Option	Type	Fonction
text	chaïne	Teste affiché dans le libellé, sur une ou plusieurs lignes
image	image	image affichée dans le widget Label. Si cette option est précisée, elle prend le dessus sur l'option text
justify	constante	justification du texte : LEFT,RIGHT ou CENTER
anchor	constante	situation du texte ou image dnas le libellé N,NE, E,SE,S,SW,W,NW ou CENTER (par défaut)
textvariable	variable	associe à une variable Tkinter (généralement StringVar) au libellé. Si la variable est modifiée, le texte du libellé est mis à jour

8 Canvas

Un canvas possède les élément prédéfinis suivants pour les dessins
⋆ line,oval (pour dessiner cercles ou ellipses)
⋆ rectangle,polygon
⋆ image ou bitmap
⋆ Text (écrire du texte)
⋆ arc (arc, corde, part de "camembert")

8.1 les lignes du canvas

On a les méthodes :
⋆ create_line (x0, y0,x1, y1, options...) : crée une ligne joignant les points de coordonnées (x0, y0) et (x1, y1)
⋆ delete (item) : détruit un élément "ligne".
⋆ create_line(x0, y0, x1 ,y1, ..., xn, yn, options) : permet de dessiner plusieurs lignes à la fois (mais elles auront alors forcément les mêmes options de dessin)
⋆ itemconfigure (item, options...) : modifie les options d'une ou de plusieurs éléments "lignes".

et quelques options : (il y en a d'autres.....)
• width : Largeur de la ligne (par défaut 1 pixel)
• fill : Couleur de la ligne (par défaut noire)

8.2 l'oval

On a les méthodes :
⋆ create_oval (x0, y0, x1, y1, options...) : crée un élément "oval" c'est à dire une ellipse qui s'inscrit dans le rectangle dont le coin en haut à gauche est (x0,y0) et le coin en bas à droite est (x1,y1). On peut donc obtenir un cercle si le dessin est fait

dans un carré
⋆ delete (item) : détruit un élément "oval".
⋆ coords (item, x0, y0) : déplace un ou plusieurs éléments "oval".

⋆ itemconfigure (item, options...) : modifie les options d'un ou de plusieurs éléments de type "oval".

8.3 Options pour les tracés

On a les options : (qui seront les mêmes pour les ovals, les rectangles et les polygones) :
⋆ fill : couleur de l'intérieur de l'objet (transparent par défaut). Si on entre une chaîne de caractères vide, l'intérieur n'est pas colorié
⋆ outline : couleur du contour de l'objet (noir par défaut) ; il n'y a pas de dessin du contour si outine =' '
⋆ width : largeur du contour (1 pixel par défaut)

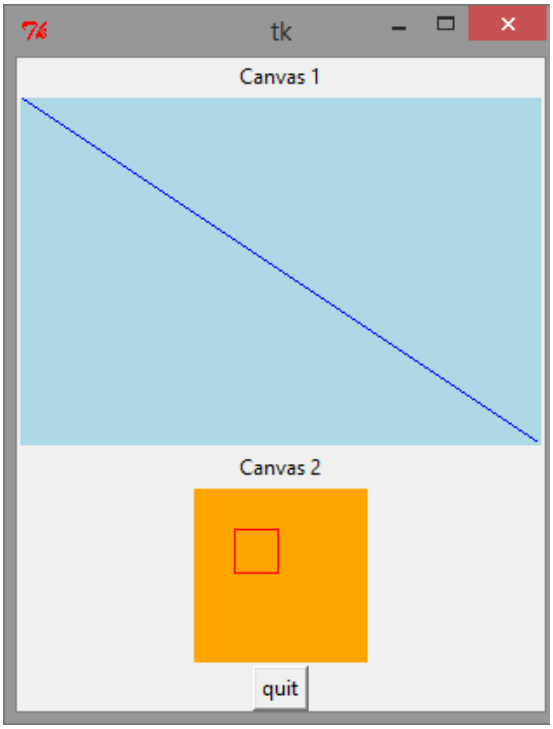
8.4 rectangles

On a les méthodes :
⋆ create_ rectangle(x0, y0, x1, y1, options...) : crée un élément rectangle dont le coin en haut à gauche est (x0,y0) et le coin en bas à droite est (x1,y1). On peut donc obtenir un carré
⋆ delete (item) : détruit un élément rectangle
⋆ coords (item, x0, y0, x1, y1) : modifie les coordonnées d'un ou de plusieurs éléments rectangle. L'argument item peut désigner un ou plusieurs éléments "rectangle" ou tout autre élément ayant les mêmes quatre coordonnées.
⋆ itemconfigure (item, options...) : modifie les options d'un ou de plusieurs élément "rectangle". (avec les options vues précédemment)

8.5 polygones

On a les méthodes :
⋆ create_polygon (xy, options...) ou create_polygon (x0, y0, x1, y1, x2, y2, x3,y3,..., xn, yn, options...) : créent un élément polygone. On doit préciser au moins 3 couples de coordonnées pour créer un nouveau polygone. On peut soit entrer directement 2n coordonnées ou mettre en paramètre une liste de coordonnées.
⋆ delete (item) : détruit un élément polygone.
⋆ coords (item, x0, y0, x1, y1, x2, y2,..., xn, yn) : modifie les coordonnées d'un ou de plusieurs éléments polygone. Notez que les coordonnées doivent être données séparément ; on ne peut pas utiliser une liste comme pour la méthode create_polygon.
⋆ itemconfigure (item, options...) : modifie les options d'un ou de plusieurs élément "polygone". (toujours avec les mêmes options)

```
from tkinter import *
Fp= Tk()
Label(Fp,text='Canvas 1').pack()
c1=Canvas(Fp,height=200,width=300,bg='light blue')
c1.pack( )
Label(Fp,text='Canvas 2').pack()
c2=Canvas(Fp,height=100,width=100,bg='orange')
c2.pack()
c1.create_line(0,0,300,200,fill='blue')
c2.create_rectangle(25,25,50,50,outline='red')
Button(Fp,text='quit',command=Fp.destroy).pack(side='bottom')
Fp.mainloop()
```



8.6 images

⋆ create_image (x0, y0, options...) : crée un élément "image" et le place à la position donnée.
Attention : Il faut que l'image ait été importée en Python : pour cela on utilise PhotoImage (voir exemple) . Il vaut mieux que les images soient au format .gif.

Exemple :
On voudrait produire une fenêtre telle que si on clique sur photo1 s'affiche la photo1, si on clique sur photo2, s'affiche la photo2, et on veut une icône python sur le bouton qui permet de quitter l'application.

```
from tkinter import *
def affiche1() : # affichage de la photo1
        can.delete(ALL)
        c=can.create_image(200,200,image=photo1)
def affiche2() :# affichage de la photo2
        can.delete(ALL)
        c=can.create_image(200,200,image=photo2)
Fp=Tk()
#importation des images en python
photo1=PhotoImage(file='laphoto1.gif')
photo2=PhotoImage(file='laphoto2.gif')
photo3=PhotoImage(file='icone.gif')
can= Canvas(Fp,height=400,width=400,bg='dark grey')
can.pack(side=LEFT)
b2=Button(Fp,text='photo1',command=affiche1)
b2.pack(side=LEFT)
b3=Button(Fp,text='photo2',command=affiche2)
b3.pack(side=LEFT)
b1=Button(Fp,image='photo3',command=Fp.destroy)
b1.pack(side=BOTTOM)
Fp.mainloop()
```



Remarques pour les écritures
On peut configurer la police, la taille, gras ou non .....
Pour cela il y a le paramètre font composé de 3 données : nom de la police, taille, aspect

Exemple :
bq=Button(calc,text='Q',font=('courier',12,'bold'),command=calc.destroy)
on peut aussi se définir un paramètre de font pour s'en resservir plusieurs fois :
Helv12=('Helvetica',12,'bold')
etiq=Label(fen,text='truc ',font=Helv12)

9 Gestion des événements

9.1 La méthode bind

La méthode bind() va permettre de lier un événement extérieur à une action.

L'événement peut être :
l'action sur une touche du clavier
un clic de souris
un mouvement de la souris

L'action associée peut être :
lancer une méthode
agir sur une fenêtre
terminer le programme

L'événement peut être associé :
à un widget
à la totalité de la fenêtre
à l'application elle même

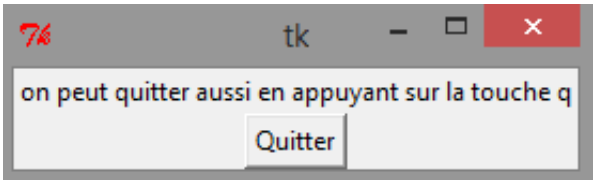
La méthode bind() permet de lier un widget, une action externe et un événement.
Elle comporte deux arguments :
la séquence, action externe qui déclenche l'événement
la fonction, méthode à exécuter lors du déclenchement de l'événement.

La méthode doit être déclarée avant son déclenchement.

```
from tkinter import*
Fp=Tk()

def stop(event) :
        Fp.destroy()

lab=Label(Fp,text='on peut quitter aussi en appuyant sur la
    touche q')
lab.pack()
Fp.bind('<q>',stop)
bout=Button(Fp,text='Quitter',command=Fp.destroy)
bout.pack()
Fp.mainloop()
```



Liste des événements

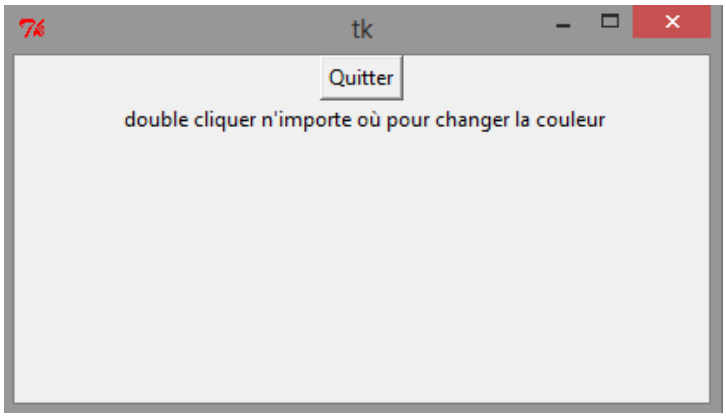
Événements liés à la souris :
<Enter> : détecte l'entrée de la souris dans un widget
<Leave> : détecte que la souris quitte un widget
<Button-1> : clic sur le bouton gauche
<Button-3> : clic sur le bouton droit
<DoubleButton1> : détecte un double click sur le bouton 1 de la souris.
<B1-Motion> : la souris se déplace avec le bouton gauche pressé
<ButtonRelease> : relâchement du bouton

Événements liés au clavier :
<Key> action sur n'importe quelle touche
<KeyPress> : détecte que l'utilisateur a tapé sur une touche
<KeyPressa> : détecte que l'utilisateur a tapé sur la touche a minuscule
<a> : détecte que l'utilisateur a tapé sur la touche a
<KeyPressA> : détecte que l'utilisateur a tapé sur la touche a majuscule
<Escape> : détecte la touche d'échappement
<Return> : action sur la touche [Enter]

Séquences combinées :
<Control>
<Shift>

Par exemple : Fp.bind('<Control-F10>',stop) ou stop est la fonction de l'exemple précédent.

```
from tkinter import*
import random
Fp=Tk()
Fp.geometry('400x200')
def changer(event) :
    L=['blue','green','red']
    couleur=random.choice(L)
    Fp.configure(bg=couleur)
bout=Button(Fp,text='Quitter',command=Fp.destroy)
bout.pack()
lab=Label(Fp,text=' double cliquer n\'importe où pour
    changer la couleur')
lab.pack()
Fp.bind('<Double-Button-1>',changer)
Fp.mainloop()
```



9.2 Exemple de gestion d'un return dans un widget entry

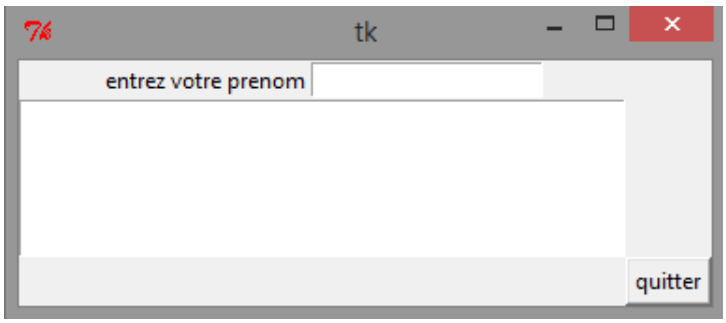
On veut qu'un retour chariot après l'écriture du texte dans la zone d'un widget entry déclenche par exemple l'écriture bienvenue et bonjour prénom dans une zone de dialogue.

```
from tkinter import *

def affiche(event) :
        p=entree.get() # pour récupérer de qui a été saisi
        txt.insert(END,'bienvenue et bonjour '+p )

Fp = Tk()
cadre=Frame(Fp)
cadre.grid(row=0,column=0)
lab_ent=Label(cadre,text="entrez votre prenom")
lab_ent.grid (row=0,column=0)
entree=Entry(cadre)
entree.grid(row=0,column=1)
txt=Text(Fp, height=5,width=40)
txt.grid(row=1,column= 0)
b1=Button(Fp,text="quitter",command=Fp.destroy)
b1.grid(row=2,column =1)

entree.bind("<Return>",affiche)
Fp.mainloop()
```



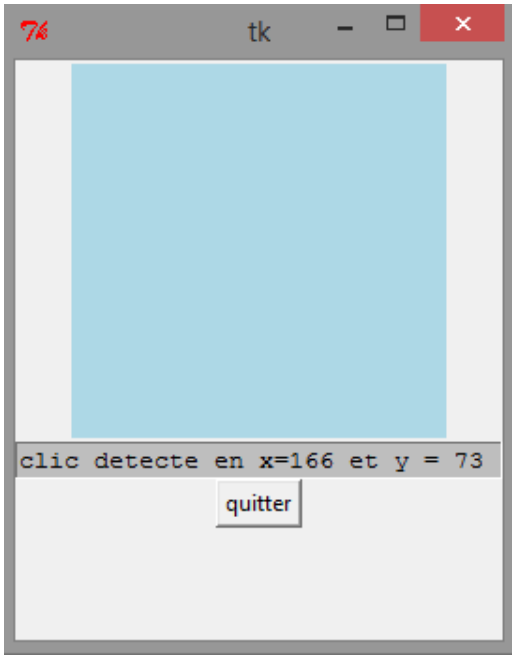
entree.bind('<Return>',affiche) : fait le lien entre la commande affiche et le retour chariot dans le widget Entry nommé entree

La fonction affiche a obligatoirement un paramètre event même si on ne s'en sert pas, comme ici.

9.3 Exemple de gestion du click souris

On veut récupérer les coordonnées d'un click de souris.

```
from tkinter import *
def donne_position(event) :
        txt.delete('0.0',END)# on efface l'écriture précédente
        txt.insert(END,'clic detecte en x='+str(event.x)+ ' et y =' +
    str(event.y))
Fp = Tk()
Fp.geometry('260x310')
cadre=Canvas(Fp,width=200, height=200, bg='light blue')
cadre.bind('<Button-1>',donne_position)
cadre.pack()
txt=Text (Fp,height=1,bg='gray')
txt.pack()
b=Button(Fp,text='quitter',command=Fp.destroy)
b.pack()
Fp.mainloop()
```



cadre.bind(.....) : associe la commande donne_position à l'évènement cliquer sur le bouton de la souris dans le widget cadre
Il y a toujours un paramètre (event) à cette fonction même si on ne s'en sert pas. Ici on s'en sert pour récupérer les coordonnées du click, coordonnées qui sont données par event.x et event.y

10 Les fenêtres fille

Pour créer une fenêtre fille, c'est à dire rattachée à une fenêtre qui la crée, il faut définir son hérédité.

```
from tkinter import *
Fp= Tk()

def creer() :
        Fp.fille=Tk()
```

```
        msg=Message(Fp.fille,text='bienvenue dans notre nouvelle fenêtre',
                                        bg='green',fg='red')
        msg.pack()
        Button(Fp.fille,text='Fermer',command=Fp.fille.destroy).pack()
Button(Fp,text='creer une fenetre',command=creer).pack()
Button(Fp,text='quit',command=Fp.destroy).pack()
Fp.mainloop()
```

La methode Toplevel
Cette méthode va nous permettre d'ouvrir de nouvelles fenêtres indépendantes.

```
from tkinter import *

def nouvfen() :
        F1=Toplevel()
        F1.geometry('400x100')
        F1.title('nouvelle fenêtre')
        Label(F1,text='ceci est une nouvelle fenêtre').pack(side='bottom')
        Button(F1,text='Fermer',command=F1.destroy).pack()

Fp= Tk()
Button(Fp,text='quit',command=Fp.destroy).pack()
Button(Fp,text='nouvelle fenêtre',command=nouvfen).pack()
Fp.mainloop()
```

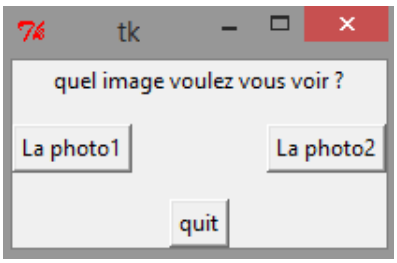
11 Afficher des images

Le widget PhotoImage permet d'afficher des images dans un Label ou un Canvas.
Le fichier image doit se trouver dans le même répertoire que le script python concerné. Seules sont reconnues les images aux formats gif ou ppm.

```
from tkinter import *

def affiche(i) :
        F1=Toplevel()
        lab=Label(F1,image=i)
        lab.pack()
        Button(F1,text='Fermer',command=F1.destroy).pack()

Fp= Tk()
Fp.geometry('200x100')
photo1=PhotoImage(file='photo1.gif')
photo2=PhotoImage(file='photo2.gif')
Label(Fp,text='quel image voulez vous voir ?').pack(side='top')
Button(Fp,text='quit',command=Fp.destroy).pack(side='bottom')
Button(Fp,text='La photo1',command=lambda  : affiche(photo1)).pack(side='left')
Button(Fp,text='La photo2',command=lambda  : affiche(photo2)).pack(side='right')
Fp.mainloop()
```



command=lambda : affiche(photo1) permet d'appeler la fonction affiche pour la variable photo1.